

0x02 Компиляция и настройка ядра. Инструменты работы с ядром.

Разработка ядра Linux



Проверить, идет ли запись

**Меня хорошо видно
& слышно?**





Степанов Вячеслав

Начальник сектора (НТЦ Модуль)

- 5 лет опыта разработки устройств на SoC Rockchip
- Опыт сопряжения линейки российских спецвычислителей на базе архитектуры NeuroMatrix с процессорами Эльбрус 8С и 8СВ (архитектура e2k)
- Участвовал в портировании ядра Linux на отечественный процессор K1879BM8Я

 @st_viacheslavik

 viacheslav.stepanov359@gmail.com



Правила занятия



Активно
участвуем



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу



Вопросы можно прислать в
телеграмм,
Отвечу на них позже

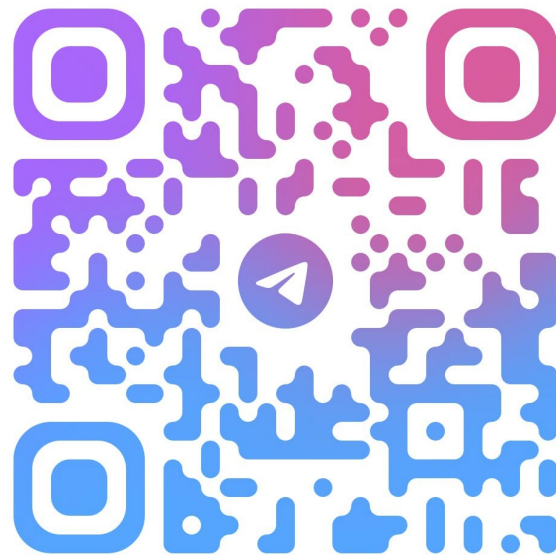


Правила вебинара



Off-topic обсуждаем
в закрытом чате в
Telegram !

Кто еще не
присоединился,
сканируйте QR код :)



Цели вебинара

К концу занятия вы сможете

скомпилировать ядро Linux.

настроить конфиг ядра без лишних модулей и опций

быстро собирать ядро на хостовой машине
используя все ресурсы вашего компьютера



Смысл

Это нужно уметь чтобы:

1. Чтобы оптимизировать систему под конкретное оборудование.

2. Чтобы включать или отключать нужные функции и модули.

3. Чтобы разрабатывать и тестировать свои модули и патчи.

4. Чтобы поддерживать уникальное или устаревшее оборудование.



Маршрут вебинара

Настройка конфигурации

Компиляция и установка

chroot

Практика



Инженерный подход

В своих объяснениях я буду опираться на инженерный подход : мы не будем углубляться в абстрактные концепции без контекста, а сосредоточимся на том, как каждая идея, каждый принцип может быть применен в реальных программно-аппаратных проектах.

Теория — это когда все известно,
но ничего не работает. Практика
— это когда все работает, но никто
не знает почему. Мы же
объединяем теорию и практику:
ничего не работает... и никто
не знает почему!



**Кто-нибудь компилировал
ядро Linux ?**



Настройка конфигурации

Автоматическая минимизация

При сборке ядра можно и нужно использовать цели из **Makefile** в исходниках ядра.

Ниже представлен список целей которые есть в **Makefile**.

```
make defconfig          #создаёт базовую конфигурацию для конкретной
платформы
make localmodconfig     #автоматически создаёт конфигурацию на основе
текущих загруженных модулей
make olddefconfig       #обновляет существующую конфигурацию до новой
версии ядра
make oldconfig          #интерактивное обновление конфигурации с
запросом новых опций
make silentoldconfig    #то же самое, что и oldconfig, но без вывода
лишней информации
```



Автоматическая минимизация

- Стандартное ядро Linux включает в себя множество драйверов
- Это как большой набор ключей, где каждый ключ предназначен для работы с определённым устройством или оборудованием

Но у каждого компьютера своё конкретное оборудование, поэтому большая часть этих драйверов просто не нужна



localmodconfig

- помогает создать **более лёгкое ядро**, оставив только те драйверы, которые действительно используются в нашей системе
- используется в процессе сборки ядра Linux для создания конфигурационного файла .config, который включает **только те модули и параметры**, которые **необходимы** для работы на **текущем оборудовании**

Это похоже на то, как если бы вы убрали из ключей все лишние, которые никогда не используете - сумка станет легче и порядок лучше



Почему это важно для сборки ядра?

Без localmodconfig:

- процесс сборки занимает больше времени
- сборка требует больше ресурсов компьютера

Это как если бы вы пытались собрать мебель по инструкции, которая включает детали для всех возможных модификаций продукта - просто излишне

С localmodconfig:

- более чистая конфигурация ядра (только то, что действительно нужно нашей системе)
- процесс сборки быстрее и эффективнее
- компилятору не приходится обрабатывать код всех лишних драйверов

```
make localmodconfig
```

Ручная корректировка

Для учебных целей мы отключаем механизмы обязательного контроля доступа (MAC), так как они не требуется в тестовой среде виртуальной машины

```
CONFIG_SECURITY_SELINUX  
CONFIG_SECURITY_SMACK  
CONFIG_SECURITY_TOMOYO  
CONFIG_SECURITY_APPARMOR  
CONFIG_SECURITY_YAMA
```



Ручная корректировка

В будущих занятиях мы планируем освоить работу с **kgdb**, что потребует предварительного отключения **KASLR**

```
CONFIG_RANDOMIZE_BASE
```



Ручная корректировка

Чтобы увеличить производительность системы имеет смысл отключить защитные механизмы для процессоров и защитные механизмы для таблицы страниц

```
CONFIG_CPU_MITIGATIONS
```

```
CONFIG_MITIGATION_SPECTRE_BHI
```

```
CONFIG_MITIGATION_RFDS
```

```
CONFIG_PAGE_TABLE_ISOLATION
```



Ручная корректировка

ZSWAP можно отключить, так как этот механизм в основном предназначен для мобильных устройств

- Основная функция – при нехватке оперативной памяти он сжимает страницы памяти и хранит их в специальной области ОЗУ вместо записи на диск
- Это обеспечивает более быстрый доступ к этим данным

`CONFIG_ZSWAP`



Ручная корректировка

Поскольку **BPF** мы не планируем использовать в нашем курсе, он рассмотрится только в рамках механизмов отладки ядра, его можно безопасно отключить

```
CONFIG_BPF
```

```
CONFIG_BPF_SYSCALL
```

```
CONFIG_BPF_JIT
```

```
CONFIG_BPF_EVENTS
```

```
CONFIG_BPFILTER
```



Ручная корректировка

В процессе выполнения домашних заданий по разработке и отладке ядра Linux требуется настройка специальных инструментов отладки.

Для эффективной диагностики проблем и анализа поведения кода необходимо включить соответствующие отладочные опции в конфигурации ядра.



Ручная корректировка

```
# поддержка debugfs
```

```
CONFIG_DEBUG_FS=y
```

```
# поддержка ftrace
```

```
CONFIG_FTRACE=y
```

```
CONFIG_FUNCTION_TRACER=y
```

```
CONFIG_DYNAMIC_FTRACE=y
```

```
CONFIG_FUNCTION_GRAPH_TRACER=y
```

```
CONFIG_STACK_TRACER=y
```

```
# поддержка KUnit
```

```
CONFIG_KUNIT=y
```

```
CONFIG_KUNIT_TEST=y
```



Ручная корректировка

```
# поддержка KASAN  
CONFIG_KASAN=y  
CONFIG_STACKTRACE=y  
CONFIG_KASAN_GENERIC=y  
CONFIG_KASAN_INLINE=y  
CONFIG_KASAN_EXTRA_INFO=y
```

```
# поддержка KGDB  
CONFIG_KGDB=y  
CONFIG_KGDB_SERIAL_CONSOLE=y  
CONFIG_DEBUG_INFO=y  
CONFIG_SERIAL_CONSOLE=y  
CONFIG_CONSOLE_POLL=y
```



Ручная корректировка

```
# поддержка Kprobe:  
CONFIG_KPROBES=y  
CONFIG_KPROBE_EVENT=y
```



Редактирование .config файла

Поправить **.config** файл можно 2-я способами:

- псевдографический интерфейс

```
make menuconfig
```

- прямое редактирование **.config** файла

```
vim .config
```

Редактирование .config файла

Оба метода настройки конфигурации ядра имеют свои особенности

- в интерфейсе **menuconfig** удобно отыскать нужную опцию для её отключения
- при использовании **vim** прямое отключение параметров может оказаться невозможным из-за того, что некоторые настройки **CONFIG_*** автоматически включаются другими параметрами конфигурации

Компиляция и установка

Компиляция ядра и его сборка в *.deb пакеты

Два основных метода компиляции ядра:

- Компиляция на одном ядре (удобна только для отладки сценария сборки)
- локальная компиляция с использованием всех доступных ядер одного процессора.

В нашем случае мы будем использовать второй подход, который предполагает эффективное распределение задачи по всем ядрам процессора на единственном компьютере.

```
make -j$(nproc) deb-pkg 2>error.log
```



Компиляция ядра и его сборка в *.deb пакеты

- Для отладки процесса сборки ядра важно правильно организовать логирование ошибок
- Когда процесс компиляции завершается с ошибкой, все сообщения об ошибках направляются в поток стандартных ошибок (**stderr**)
- Чтобы сохранить эти важные данные для последующего анализа, можно перенаправить **stderr** в файл **error.log**



Установка deb-пакетов ядра

В `/usr/src/` появятся `*.deb` пакеты, их можно установить используя `dpkg`.

```
dpkg -i linux-image-6.1.130_*_amd64.deb
```

```
dpkg -i linux-headers-6.1.130_*_amd64.deb
```

```
dpkg -i linux-libc-6.1.130_*_amd64.deb
```

```
reboot
```



Особенности сборки ядра на виртуальной машине

Основной недостаток сборки ядра в среде виртуальной машины – существенное **увеличение времени компиляции** по сравнению со сборкой непосредственно на физической машине:

- Дополнительный уровень абстракции между процессом сборки и аппаратным обеспечением
- Замедляется выполнение операций

В связи с этим ниже представлен более эффективный подход к сборке ядра, который позволяет значительно сократить время компиляции



Альтернативный Метод: Сборка локально через chroot

Использование **chroot** для сборки ядра Linux

- обеспечивает **создание изолированного окружения** непосредственно на **локальной машине** разработчика
- существенно **ускоряет процесс компиляции** за счет прямого доступа к аппаратным ресурсам без дополнительных слоев виртуализации



Подготовка окружения chroot

Создать корневую директорию для сборки на хосте

```
mkdir -p /home/dir_chroot
```

Скопируйте исходники ядра из виртуальной машины в /home/dir_chroot

```
scp -r linux-* <ваш пользователь>@<ваш ip>:.
```

```
# пример использования scp, копируем linux-6.1.130 в home  
директорию пользователя host с ip 192.168.122.1
```

```
scp -r linux-6.1.130 host@192.168.122.1:.
```



Подготовка окружения chroot

Для инициализации окружения **chroot** рекомендуется использовать предварительно подготовленный скрипт

Можете разобрать его построчно самостоятельно



Подготовка окружения chroot 1

```
#!/bin/bash
```

```
chr="/home/dir_chroot"
```

```
sudo debootstrap \
```

```
--include ncurses-term,mc,locales,nano,gawk,lsb-release,acl \
```

```
--components=main,contrib,non-free \ #дополните флаги
```

```
репозитория
```

```
${chr} \
```

```
# укажите репозиторий откуда будут скачиваться пакеты
```

```
cp /etc/resolv.conf ${chr}/etc/resolv.conf
```

```
echo "cp /etc/resolv.conf ${chr}/etc/resolv.conf"
```



Подготовка окружения chroot 2

...

```
cd ${chr}
mount proc -t proc ./proc
echo "proc"
mount sys -t sysfs ./sys
echo "sys"
mount --bind /dev ./dev
echo "dev"
```



Переход в chroot

```
cd /home/dir_chroot  
chroot .
```

После выполнения этих команд система начинает считать текущую директорию новым корневым каталогом **/**.



Установка зависимостей для сборки ядра

Установим минимальный набор пакетов для сборки ядра, остальные пакеты можно доустановить выполняя команду

`make -j$(nproc) deb-pkg`

```
apt install -y build-essential libncurses-dev  
bison flex libssl-dev bc rsync
```





LIVE

Вопросы для проверки

1. Почему сборка в chroot быстрее?
2. Как создать конфигурацию для вашего устройства по умолчанию
3. Какой компилятор чаще всего используют для компиляции ядра?



Вопросы для проверки

1. Сборка в chroot быстрее, потому что среда изолирована и минимальна, что снижает влияние внешних процессов и ускоряет доступ к ресурсам.
2. `make savedefconfig`
3. Чаще всего для компиляции ядра используют GCC, но также возможны Clang/LLVM и другие компиляторы, поддерживающие архитектуру.



Вопросы



если есть вопросы



если вопросов нет

Подведём итоги занятия



**Отправьте в чат эмодзи, который
отражает ваше настроение
после занятия**



**Продолжите высказывание: теперь
я умею/знаю...**



**Что планируете использовать
в работе?**

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

Приглашаем на следующий вебинар: “0x03 Ядро Linux. Обзор ядра Linux и его архитектуры. Основные функции ядра.”



Ссылка на вебинар будет
в ЛК за 15 минут



Материалы
к занятию в ЛК —
можно изучать



Обязательный материал
обозначен красной лентой

OTUS

